

DISRUPT! Claude-A-Thon

Date: 27th February 2026

Time: 9:30 AM – 1:30 PM

Our Mission: Learn 2 Lead | Get the Accelerated Impact

Welcome to a one-day, hands-on sprint designed to fundamentally change the way you write code. Our objective is to move beyond traditional programming and introduce you to a smarter, more powerful way of building software — where you and Claude Code work as true collaborators. More than just vibe coding, Claude Code is an agentic AI pair programmer that understands context, navigates your codebase, executes tasks end-to-end, and helps you tackle real engineering challenges with precision and speed.

This hackathon will teach you to harness the full power of Claude Code across the entire development lifecycle — from scaffolding projects and writing complex logic, to debugging, refactoring, running terminal commands, and shipping production-ready code. You'll learn to think in terms of intent and outcomes, letting Claude Code handle the heavy lifting while you stay focused on what matters most.

Your AI Toolkit

You'll get hands-on experience with the core capabilities of Claude Code:

1.  **Conversational Coding:** Talk to Claude Code in plain English — ask questions, brainstorm ideas, get explanations, and generate code from scratch through a natural back-and-forth dialogue.
2.  **Targeted Edits:** Precisely refactor, modify, and improve specific files or functions with full context-awareness — Claude Code understands your codebase, not just the snippet in front of it.
3.  **Agentic Execution:** Give Claude Code a high-level goal and watch it reason, plan, and autonomously execute complex, multi-step tasks across your entire codebase — including running terminal commands, managing files, and iterating on its own output.
4.  **Extended Thinking:** Enable Claude to reason through complex problems step-by-step before responding — ideal for architectural decisions, debugging complex issues, and multi-constraint optimisation.
5.  **Tool Use / MCP:** Define tools with JSON schemas for Claude to call, and integrate external data sources and APIs using the Model Context Protocol (MCP) — the preferred way to connect Claude to external systems.
6.  **Vision / Multimodal:** Upload wireframes, mockups, or screenshots for Claude to analyse visually and generate matching code or structure.

7.  **Plugins:** Extend Claude Code's capabilities by connecting it to external tools and services — such as databases, APIs, or developer utilities — directly within your session. Plugins let Claude take actions beyond the codebase, like querying live data or triggering external workflows.
8.  **Hooks:** Automate Claude Code's behaviour at key lifecycle moments — before or after a file edit, on session start, or when a command runs. Hooks let you enforce standards, trigger linting, auto-run tests, or log activity without manual intervention.
9.  **Skills:** Package reusable, task-specific prompt workflows as Skills — predefined sequences Claude can invoke on demand. Think of Skills as macros for common engineering tasks: "run security audit", "generate API docs", or "create unit tests for this module."

The Challenge: CRUD Dashboard for Managing Inventory Items

Your mission is to build a simple yet powerful **CRUD (Create, Read, Update, Delete) Dashboard** for managing inventory items. You'll create a user-friendly web interface where users can track, update, and manage stock items, view inventory levels, and ensure operational efficiency. This project is the perfect playground to test every feature in your AI toolkit.

Hackathon Agenda & Missions

Time	Mission	Primary Claude Code Feature & Task Mapping
	Pre-Mission: Base Camp	Get your environment ready. Always start a Claude Code session by loading your SPEC and existing files. Pre-requisites: (1) Clone your Hackathon Repo: https://github.com/Cognizant-Learning/VibeCoding-CRUD-Dashboard.git (2) Open the project folder in your terminal (3) Start a Claude Code session: <code>claude</code> (4) Load your project brief by referencing the <code>projectbrief.md</code> file in your opening message (5) Confirm Claude has understood the project structure before proceeding
15 mins	Mission 1: Camp Setup & Compass Calibration  	Launch the Claude Code REPL and establish your base rules. Use <code>/init</code> to bootstrap project memory, then ensure your repo has a clear CLAUDE.md (project instructions Claude loads every session). Optionally set your session model with <code>/model</code> .
25 mins	Mission 2: The Cartographer's Recon (Plan Mode → SPEC only)  	Enter Plan Mode (read-only safety). In this mode Claude can explore, read, grep, and propose a plan without editing files — perfect for translating

		Projectbrief.md into a solid blueprint covering all CRUD entities, data models, and UI structure.
30 mins	Mission 3: The Quartermaster's Runbook (Plan Mode → BUILD PLAN only) 	Still in planning mindset, forge the "quest steps": create Plan.md with phased tasks + per-task "done means" checks — covering entity creation, API routes, dashboard UI, and validation logic. Use /compact if the session grows and you need to compress context while keeping key decisions.
45 mins	Mission 4: The Forge of Many Hands (Agentic Build + Subagent Crew) 	Switch out of Plan Mode and let Claude execute . Build incrementally feature-by-feature — backend CRUD endpoints first, then the dashboard UI, then data bindings. Use Checkpointing as your "time-turner": /rewind (or Esc Esc) to restore code and/or conversation when things drift. Then unleash Subagents via /agents : create or use specialist agents (e.g., Explorer, Tester, Doc-scribe, Security Scout) so tasks run in parallel with isolated context.
30 mins	Mission 5: The Wizard's Encore (Creative Freedom) 	Now you're the inventor: add one or two standout enhancements — e.g., low-stock alerts, bulk import/export, inventory analytics charts, role-based access, or search and filter functionality. Spawn a dedicated innovation subagent to brainstorm options without polluting the main thread. Keep it shippable by relying on /rewind when experiments go sideways.
20 mins	Mission 6: The Artificer's Arsenal (Plugins, Hooks & Skills) 	Now supercharge your workflow. Plugins : Connect Claude Code to an external tool or data source relevant to your app — e.g., a live stock data API via MCP, a database plugin, or a CSV import utility. Hooks : Wire up at least one automation hook — e.g., auto-run linting after every file save, trigger unit tests post-edit, or log all Claude-generated changes to a changelog. Skills : Package one reusable workflow as a named Skill — e.g., a "generate CRUD test suite" or "audit inventory schema" skill that can be invoked on demand in future sessions. Document all three in your prompts.md, including what each does and why you chose it.

Project Naming Convention

Participants should create their folder name in the following pattern:

usecasename-teamname-empid1-empid2-empid3

Example: CRUDDashboard-POD1-987345-465747-234587

*Submission URL: Use the GitHub URLs below based on location to submit the score. **DO NOT SUBMIT TO A DIFFERENT FOLDER APART FROM YOUR LOCATION OR A DIFFERENT GITHUB URL***

Chennai: <https://github.com/Cognizant-Learning/claude-code-hackathon-spe-feb-2026/tree/main/Solutions-code-base-Chennai>

Pune: <https://github.com/Cognizant-Learning/claude-code-hackathon-spe-feb-2026/tree/main/Solutions-code-base-Pune>

Kolkata: <https://github.com/Cognizant-Learning/claude-code-hackathon-spe-feb-2026/tree/main/Solutions-code-base-Kolkata>

Judging Criteria

Feature Implementation, Code Quality + Agentic Architecture — 50 Points

Does the application solve the CRUD use case end-to-end (Create, Read, Update, Delete all functioning correctly)? Is the code well-structured and secure? Are agentic and multi-step patterns used appropriately — tools well-defined, stable flows, error handling where needed? Evidence of disciplined use of Plan Mode, incremental builds, agents, subagents, and checkpointing will be assessed here.

Prompt Engineering Effectiveness + Claude Feature Utilisation — 30 Points

Crafting prompts that go beyond simple instructions — articulating high-level intent, providing rich context, using examples, and guiding Claude Code through qualitative feedback to steer outcomes. Assessed on the quality and structure of prompts (Instruction, Context, Example, Cue) as documented in prompts.md. Evidence of using the right Claude capabilities for the right tasks (e.g., model tiering, tool use, MCP, extended thinking, prompt structuring).

Plugins, Hooks & Skills Utilisation — 10 Points

Did the team meaningfully use Claude Code's extensibility features? Assessed on whether a Plugin was configured and used, a Hook was defined and triggered, and a Skill was created and documented — with clear evidence of purpose and outcome in prompts.md.

Creativity and Additional Features — 20 Points

Did you push beyond the brief and create something that truly showcases the power of agentic AI development with Claude Code? Recognition for innovative use of Claude's autonomous capabilities, unexpected features, or standout engineering decisions.

Submission Checklist

Before submitting, ensure your project folder contains:

- ☐ projectbrief.md — original brief (provided, do not modify)
- ☐ CLAUDE.md — project-level Claude Code instructions
- ☐ requirements.md — extracted and refined requirements
- ☐ Plan.md — phased build plan with done-means checks
- ☐ prompts.md — complete log of all prompts across all missions, including a dedicated Plugins/Hooks/Skills section
- ☐ Plugin configuration file
- ☐ Hook definition file
- ☐ Skill file
- ☐ Project source code — all application files
- ☐ App runs locally without errors

Folder naming: CRUDDashboard-[teamname]-[empid1]-[empid2]-[empid3] *Example:*
CRUDDashboard-sunrocks-987345-465747-234587

Good luck, build boldly, and let Claude Code surprise you. 🚀